

ENGINEERING CASE STUDY

Distributed Usage Metering and Billing Ledger

Role: Head of Engineering. Time: 24 hours from receipt. *Build the correctness core airtight and make deliberate, well argued cuts elsewhere.*

EXPECTED EFFORT: INTENTIONALLY LARGER THAN 24 HOURS
WE DO NOT EXPECT EVERYTHING DONE

CONTEXT

The Problem

We run systems where money like state must stay correct under high volume, partial failure, retries, and occasionally hostile input. We want to see how you architect for correctness under those conditions; and how you work, since we expect you to use AI agents heavily and to move fast into unfamiliar territory. Use any agents and tools you would normally use; we are as interested in how you build as in what you ship.

WHAT TO BUILD

System Overview

A usage metering and billing system for a multi tenant SaaS product, split into two independently running services that communicate over an at least once message channel. You choose the channel: table backed queue, Redis, NATS, Kafka. It must redeliver on failure, and you should assume it will.

SERVICES

SERVICE	RESPONSIBILITY
Ingest	Accepts usage events from tenants and from an external usage provider via webhook, then publishes them onto the channel.
Ledger	Consumes events, rates them, and writes balanced double entry postings; serves balances and statements.

OFFICE

106 West 56th Street, 7th Floor
New York, NY 10019

CONTACT

argonav.ai
hello@argonav.ai

PARENT

A Keel Merchant
Bank Company

ENDPOINTS

API Surface

Endpoints are small on purpose. Depth is the point.

INGEST SERVICE

METHOD	PATH	DESCRIPTION
POST	/events	Tenant authenticated usage event: {tenant, metric, quantity, idempotency_key}
POST	/webhooks/usage	Usage from external provider, HMAC signed. Verify signatures, reject tampered or replayed deliveries, tolerate retries.

LEDGER SERVICE

METHOD	PATH	DESCRIPTION
GET	/balance	Tenant scoped read.
GET	/statement	Tenant scoped read.
POST	/adjustments	Credit or manual adjustment. Admin only.
POST	/periods/close	Finalize a billing period for a tenant. Safe under concurrent invocation.

Pre seed two or three tenants plus one admin credential; no need to build signup. Define a simple price book for rating.

INVARIANTS

What Must Hold

These invariants must hold under concurrency, retries, and crashes.

CORE INVARIANTS

INVARIANT	REQUIREMENT
Zero sum	Every transaction's postings net to zero; the ledger is append only and never edited in place.
Exactly once	Because the channel is at least once, the same usage event redelivered by the queue, retried by a client, or replayed by the webhook caller must produce its charge exactly once. Define and defend your dedup boundary.

OFFICE

106 West 56th Street, 7th Floor
New York, NY 10019

CONTACT

argonav.ai
hello@argonav.ai

PARENT

A Keel Merchant
Bank Company

INVARIANT	REQUIREMENT
Crash safety	If either service is killed at any point mid processing and restarted, every invariant still holds. No lost events, no double posts, no partially applied transactions, no orphaned state. We will test this by killing your processes mid run.
Tenant isolation	No tenant can read or affect another's data, ever.
No drift	Any derived balance equals the sum of its underlying postings. Money is exact integer minor units; no floating point.
Authorization	Privileged actions require elevated authorization, not merely a valid tenant token.
Immutable close	A closed period cannot be re closed or mutated; concurrent close attempts resolve to a single close.
Webhook integrity	The webhook boundary rejects forged, tampered, and replayed deliveries.

RECONCILIATION

Background Job

A background process or a POST /reconcile admin endpoint that independently re derives each tenant's balance from the postings and flags any drift between expected and recorded state. It must detect real corruption and not raise false positives during normal concurrent operation. We will inject corruption and watch whether it catches it.

DELIVERABLES

What to Submit

REQUIRED

ITEM	DESCRIPTION
Git repository	Full history preserved. Do not squash. We will read your commit history.
Docker compose	Both services runnable with one documented command.
Automated tests	For the invariants, including at least one crash restart test and one concurrency test.
DESIGN.md	Three pages or fewer: architecture and why; how each invariant survives concurrency, retries, and crash; your exactly once dedup design and its boundary; threat model for the webhook; and an explicit what I cut and why under the time limit section.

OFFICE

106 West 56th Street, 7th Floor
New York, NY 10019

CONTACT

argonav.ai
hello@argonav.ai

PARENT

A Keel Merchant
Bank Company

ITEM	DESCRIPTION
NOTES.md	Short: how you worked, what you delegated to agents, where you had to learn something new, and where the agents led you wrong and you caught it.

OUT OF SCOPE

Do Not Build

Any UI, real payment processor integration, SSO or signup, autoscaling or HA, multi region, cloud deploy. A single node per service is expected. Distributed here means across the two services and the channel, not across machines.

EXPECTATIONS

How We Evaluate

Use any stack you are fluent in. You have 24 hours from receipt. The scope is intentionally larger than the time allows; we expect deliberate triage, not completeness.

A narrow, crash safe, exactly once core with an honest list of what you deferred beats a feature complete system that corrupts under load. We weight correctness and security far above feature count.

Your submission will undergo adversarial correctness, concurrency, and fault injection testing. If you run out of time, an honest note in DESIGN.md about a known gap is worth more than a hidden one.

OFFICE

106 West 56th Street, 7th Floor
New York, NY 10019

CONTACT

argonav.ai
hello@argonav.ai

PARENT

A Keel Merchant
Bank Company